

# Multi-Source Candidate Data Transformer — Design Note

Kovvuri Rishi Kesav Reddy | rishikesav2005@gmail.com | Eightfold Engineering Intern Assignment

## What I'm building

Eightfold pulls candidate data from disconnected sources that don't agree on field names, formats, or sometimes even basic facts about the same person. I'm building a pipeline that takes in two of those sources, figures out which records belong to the same candidate, resolves whatever they disagree on, and produces one canonical profile per candidate with a record of where every value came from and how confident I am in it. The output shape itself is configurable at runtime, so the same engine serves different consumers without me touching the code.

## Pipeline

*extract* → *normalize* → *match candidates* → *resolve conflicts + score confidence* → *build canonical profile* → *project to requested config* → *validate* → *output*

Extract and normalize happen per source, independently. Everything from "match" onward operates on the merged view across sources. I'm keeping the canonical profile completely separate from the projection step — the canonical record never changes shape no matter what config comes in; only the final projected JSON does. This was the main thing I wanted to get right structurally, since it's called out explicitly as a requirement.

## Sources I'm handling

- ATS JSON export (structured) — its own field names, doesn't map 1:1 onto my schema, so this is mostly a remapping + type-coercion job.
- Recruiter notes .txt (unstructured) — free text, no fixed structure. I pull out email, phone, and skill mentions with regex and a few heuristics around lines like "Skills:" or "Notes:". Anything I can't confidently extract, I leave out rather than guess.

I picked these two over the others mainly for time — CSV/JSON parsing and regex-based extraction are both things I can get fully correct and well-tested by tomorrow evening, versus spending hours on API auth (GitHub/LinkedIn) or PDF layout quirks (resumes) for the same grading credit.

## Schema and normalization

Using the schema from the brief as-is. The normalization choices I'm locking in:

- Phones → E.164, using the phonenumbers library. If it can't resolve a region, the value stays null rather than me assuming a country code.
- Dates → YYYY-MM. Anything not cleanly parseable becomes null.
- Country → ISO-3166 alpha-2 via a small lookup table for the common name variants I expect to see in the sample data.
- Skills → lowercase + trim, then run through a small synonym table (e.g. "js" → "javascript", "reactjs" → "react"). Anything not in the table passes through as-is but gets a lower confidence score, since I can't vouch for it being canonical.

## Matching candidates across sources

Match key, in priority order: normalized email exact match, then normalized phone exact match, then a fuzzy match on name + current company as a last resort. The first two are near-certain matches; the fuzzy fallback is a weaker signal, so I flag those matches with reduced confidence since name spelling differences don't always mean a different person.

## Resolving conflicts and confidence

When two sources disagree on a field, I use a priority ranking by field type rather than one global rule, since "most trustworthy source" isn't the same for every field:

| Field                  | Priority order (highest wins)                                             |
|------------------------|---------------------------------------------------------------------------|
| name, email, phone     | ATS JSON > notes                                                          |
| current_company, title | ATS JSON > notes                                                          |
| skills                 | Union of both sources, not a single winner — confidence per skill instead |

Confidence is a simple additive score, not anything learned: start from a base score per source (ATS higher than notes, since it's structured and presumably reviewed), add a bonus if both sources agree on the value, subtract a penalty if they conflict, and cap between 0 and 1. I picked simple-and-explainable over anything fancier on purpose — every confidence number needs to be traceable to a one-line reason, and that's part of what's being graded.

## Handling the runtime config

The config (per the example in the brief) selects fields, remaps paths via "from", sets per-field normalization, toggles provenance/confidence, and sets an on\_missing policy. My projector reads the canonical profile (never the raw sources) and walks each field's "from" path against it: null keeps the key with a null value, omit drops it, error fails the whole projection rather than returning a partially-wrong object. The result is then validated against a schema built from the config itself before it's returned — so a bad config produces a clear error, not silently wrong output.

## Edge cases I'm explicitly handling

- One source file missing entirely → pipeline still runs; its fields stay null with no provenance entry, nothing crashes.
- Malformed JSON or a broken CSV row → that record is skipped and logged, the rest of the run continues.
- Two sources disagree on email → ATS wins per the priority table, but confidence drops and both raw values stay in provenance.
- Phone number with no country code and nothing to infer it from → stays null rather than guessing a country.
- Config asks for a field path that doesn't exist on the canonical schema → fails validation at projection time, not a silent null — a typo'd path shouldn't look like a real empty field.

## What I'm consciously leaving out

Given the time I have, I'm skipping ML-based entity resolution and NLP extraction from free text — reasonable upgrades later, but they add nondeterminism and fight the "explainable" requirement. I'm also skipping GitHub/LinkedIn/resume extractors this round and keeping the CLI bare, since the brief marks that surface as lower priority than the engine.